
**COMSATS University Islamabad,
COMSATS Road, off GT Road, Sahiwal, Pakistan**



Assignment # 1

Submitted By

HABIB UR REHMAN

Roll No

FA23-BCS-251

Course

Artificial Intelligence

Course Instructor

Ms. Kanwal Fatima

Bachelor of Science in Computer Science (2023-2027)

Question 1:**CLO 6**

Students are required to apply web scraping to collect relevant information from a website, process this data for a practical use case, and develop an interactive front-end for user interaction.

The first step is to perform web scraping using Python libraries such as BeautifulSoup or Selenium to extract useful information from a chosen website. The extracted data should be processed and stored in a structured format. The second step is to utilize the extracted data in a meaningful application. The application should perform some analysis, comparison, or decision-making based on the scraped data. For example, students can extract admission eligibility criteria from a university website and compare it with user-provided academic marks to determine eligibility, or extract real-time weather data and display personalized weather alerts. The final step is to develop a front-end interface using Streamlit, Flask, or Django. The interface should allow users to interact with the data, input necessary details, and view the processed results in a user-friendly manner.

Solution:

```
import requests

from bs4 import BeautifulSoup

import streamlit as st

def get_weather(city):
    try:
        city = city.replace(" ", "-")
        url = f"https://www.timeanddate.com/weather/pakistan/{city.lower()}"
        response = requests.get(url, headers={"User-Agent":
        "Mozilla/5.0"})

        if response.status_code != 200:
            return None, None

        soup = BeautifulSoup(response.text, "html.parser")
        temp_tag = soup.find("div", class_="h2")
        temp = temp_tag.text.strip() if temp_tag else None
        condition_tag = soup.find("p")
```

```

        condition = condition_tag.text.strip() if condition_tag else
"N/A"

        return temp, condition

    except Exception as e:

        return None, f"Error: {e}"

st.title("☀️ Live Weather Checker")
City = st.text_input("Enter a city in Pakistan:", "Faisalabad")
if st.button("Check Weather"):

    temp, condition = get_weather(City)

    if temp:

        st.success(f"Current weather in **{City.capitalize()}**")
        st.write(f"Temperature: **{temp}**")
        st.write(f"Condition: **{condition}**")

    else:

        st.error(f"Could not fetch weather for {City}. Please try another
city.")

```



Live Weather Checker

Enter a city in Pakistan:

karachi

Check Weather

Current weather in **Karachi**

Temperature: 29 °C

Condition: **Scattered clouds.**

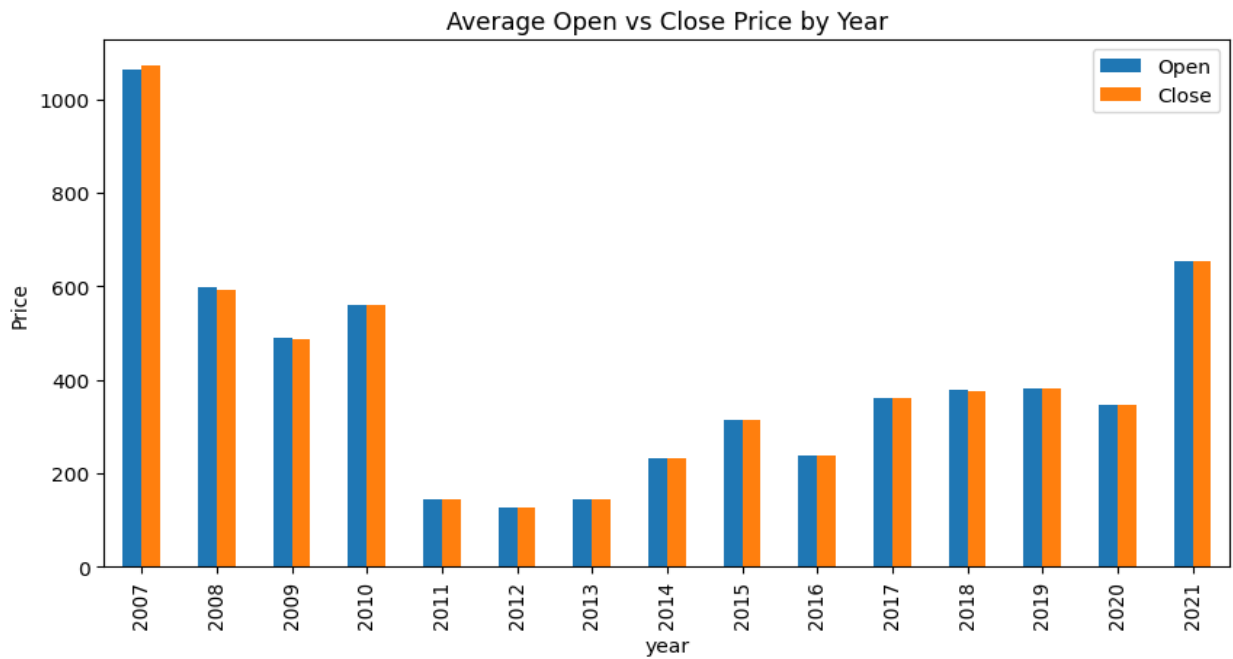
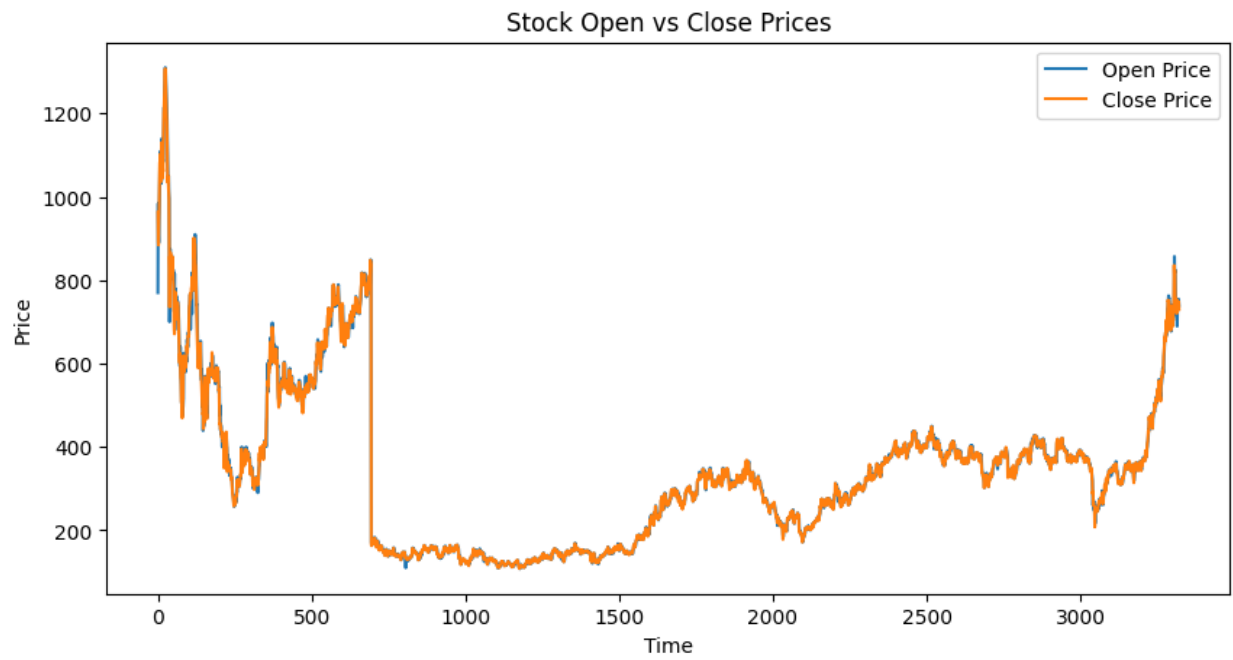
Question 2:**CLO 6**

Write Python code to draw the following graph using matplotlib (or any other relevant library):

Solution:

```
import numpy as numpy
import pandas as pd
import matplotlib.pyplot as plt
plt.figure(figsize=(10,5))
plt.plot(df['Open'], label='Open Price')
plt.plot(df['Close'], label='Close Price')
plt.title('Stock Open vs Close Prices')
plt.xlabel('Time')
plt.ylabel('Price')
plt.legend()
plt.show()

# Convert to datetime first
df['Date'] = pd.to_datetime(df['Date'])
# Extract only year
df['year'] = df['Date'].dt.year
df['year'] = pd.to_datetime(df['Date']).dt.year
avg = df.groupby('year')[['Open', 'Close']].mean()
avg.plot(kind='bar', figsize=(10,5))
plt.title("Average Open vs Close Price by Year")
plt.ylabel("Price")
plt.show()
```



Question 3

```
import streamlit as st
import numpy as np
import random
import time

def check_winner(board):
    """Check if someone won the game"""
    # Check rows
    for row in board:
        if row[0] == row[1] == row[2] != 0:
            return row[0]
    # Check columns
    for col in range(3):
        if board[0,col] == board[1,col] == board[2,col] != 0:
            return board[0,col]

    # Check diagonals
    if board[0,0] == board[1,1] == board[2,2] != 0:
        return board[0,0]
    if board[0,2] == board[1,1] == board[2,0] != 0:
        return board[0,2]

    return 0 # No winner

def is_board_full(board):
    """Check if board is full"""
    return not np.any(board == 0)

def get_empty_positions(board):
```

```

    """Get all empty positions on the board"""
    return list(zip(*np.where(board == 0)))

def computer_move(board):
    """Computer makes a random move"""
    empty_spots = get_empty_positions(board)
    if empty_spots:
        return random.choice(empty_spots)
    return None

# Initialize the game
if 'board' not in st.session_state:
    st.session_state.board = np.zeros((3,3), dtype=int)
    st.session_state.game_over = False
    st.session_state.winner = 0
    st.session_state.message = "Your turn! Click any cell to play."

# Page setup
st.set_page_config(page_title="Tic-Tac-Toe", page_icon="🎮")
st.title("🎮 Tic-Tac-Toe Game")
st.markdown("---")

# Game info
col1, col2 = st.columns(2)
with col1:
    st.markdown("**You:** 🛑 (Cross)")
with col2:
    st.markdown("**Computer:** 🟠 (Circle)")

```

```

st.markdown("---")

# Game board
board = st.session_state.board

# Create 3x3 grid
for i in range(3):
    cols = st.columns(3)
    for j in range(3):
        with cols[j]:
            # Determine what to show in the cell
            if board[i,j] == 1:
                symbol = "✖"
            elif board[i,j] == 2:
                symbol = "◯"
            else:
                symbol = " "

            # Create button for each cell
            if st.button(symbol, key=f"btn_{i}_{j}",
                           disabled=(board[i,j] != 0 or
st.session_state.game_over),
                           help=f"Row {i+1}, Column {j+1}"):
                # Player makes move
                if board[i,j] == 0 and not st.session_state.game_over:
                    board[i,j] = 1 # Player is 1 (✖)

                # Check if player won
                winner = check_winner(board)

```

```

        if winner == 1:
            st.session_state.winner = 1
            st.session_state.message = "🎉 You Won!
Congratulations!"

            st.session_state.game_over = True
        elif is_board_full(board):
            st.session_state.message = "🤝 It's a Draw! Good
game!"

            st.session_state.game_over = True
        else:
            # Computer's turn
            st.session_state.message = "Computer is
thinking..."

            # Small delay for better UX
            time.sleep(0.5)

            # Computer makes move
            comp_move = computer_move(board)
            if comp_move:
                board[comp_move[0], comp_move[1]] = 2 #
Computer is 2 (○)

            # Check if computer won
            winner = check_winner(board)
            if winner == 2:
                st.session_state.winner = 2
                st.session_state.message = "🤖 Computer
Won! Try again!"

                st.session_state.game_over = True
            elif is_board_full(board):

```

```

        st.session_state.message = "🤖 It's a
Draw! Good game!"

        st.session_state.game_over = True
    else:
        st.session_state.message = "Your turn!
Click any empty cell."

    st.rerun()

# Game status
st.markdown("---")
st.subheader(st.session_state.message)

# Reset button
if st.button("🔄 New Game", type="primary"):
    st.session_state.board = np.zeros((3,3), dtype=int)
    st.session_state.game_over = False
    st.session_state.winner = 0
    st.session_state.message = "Your turn! Click any cell to play."
    st.rerun()

# Game statistics (optional)
if st.session_state.game_over:
    st.markdown("---")
    if st.session_state.winner == 1:
        st.success("🏆 Victory! You beat the computer!")
    elif st.session_state.winner == 2:
        st.error("💻 Computer wins this round!")

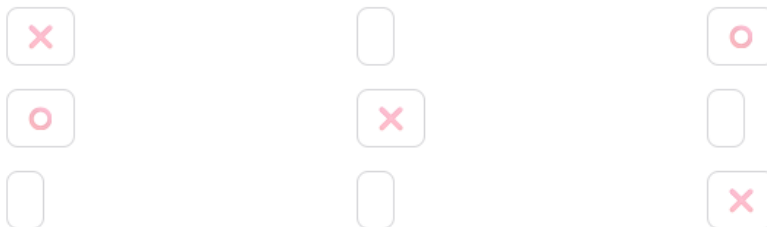
```

```
else:st.info("🤝 Draw - Nobody wins this time!")
```

Tic-Tac-Toe Game

You:  (Cross)

Computer:  (Circle)



 **You Won! Congratulations!**

 New Game

 Victory! You beat the computer!

Source code:

Click link for Code and Resources on GitHub: [Click Here](#)